

Sustainable AI Data Compression: Profiling BOA on BEDE

Measuring the energy and carbon footprint of BOA Constrictor, a neural lossless compressor for particle-physics data

Hanzila Hussain

Supervised by: Caterina Doglioni



The University of Manchester

Background

- **The growing nature of data:**
 - High Energy Physics (HEP) experiments such as the LHC record several petabytes of data every week
 - Storing it comes at a significant cost, in both budget and environmental resources
- **BOA Constrictor:**
 - Mamba based lossless compressor developed by researchers at the University of Manchester
- **The limitation:**
 - ML-based compression is generally more computationally intensive than standard algorithms like ZSTD and LZMA
 - This raises the question of where the **break-even point** lies between the carbon cost of training and running it, and the carbon saved on the disk storage it displaces.
- **Project:**
 - Build a learning guide for running BOA with CodeCarbon
 - Compare BOA's carbon footprint with classical compressors
 - Measure how much the choice of GPU changes the cost

Measuring carbon

Carbon Estimation framework:

Total carbon cost = Operational carbon + embodied carbon

We only measure operational term

$$C = E \times I$$

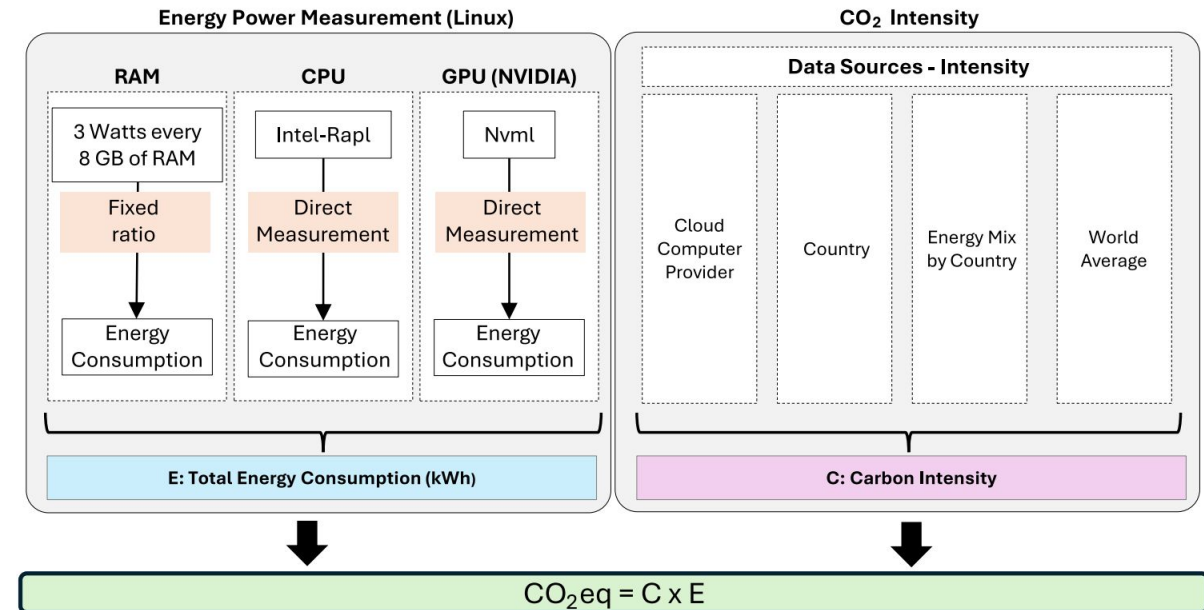
carbon = energy the code used × how dirty the grid is

What CodeCarbon measures

- GPU: board power read from the driver (NVML)
- CPU: RAPL counters, or a TDP table when virtualized
- RAM: estimated from installed capacity

What it assumes

- I: UK grid intensity, looked up offline (GBR)
- Operational energy only - embodied carbon excluded



Measured per stage: Training (per epoch), compression and decompression. Each block sits in one track() context manager, with CUDA synchronised at both ends and power sampled every second.

Data & Method

The data: CMS_DATA_float32.bin which is a flat float32 array of reconstructed jets from CERN open data.

Boa model fixed: mambav1 backbone, d_model 64, 2 layers, seq_len 10,000, batch 5, fp32, 2 KB coding chunks, UK grid intensity.

Bede (N8CIR)

4 x Tesla V100-SXM2-32GB
POWER9, 128 threads, 519 GB RAM

Workstation

1 x NVIDIA RTX 5090
Intel Core Ultra 7 265K, 94 GB

Google Colab

1 x Tesla T4
Xeon @ 2.00 GHz, 12.7 GB

What each run records

- A 30 s idle baseline, then the energy of every training epoch (3 epochs per size)
- Compression and decompression at the final epoch, each checked bit-for-bit with SHA-256
- One CSV row per stage: GPU/CPU/RAM energy split, kWh per GB, MB/s and a hardware fingerprint

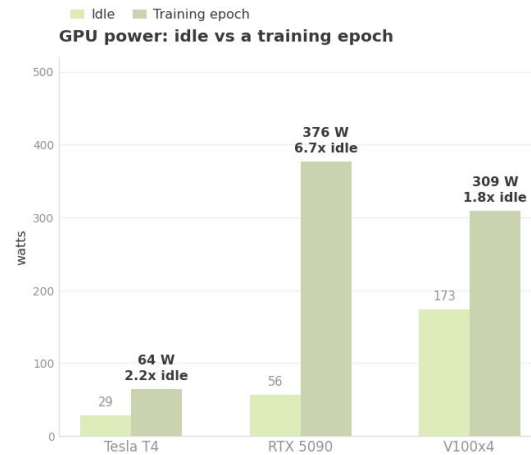
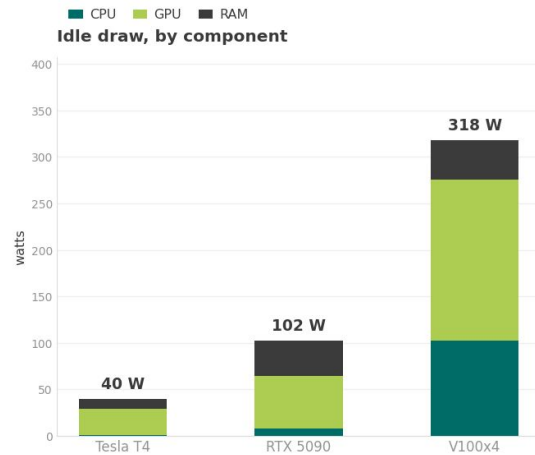
Scaling the input

- Seven input sizes per machine: 10, 25, 50, 100, 250, 500 and 1000 MB
- The CMS file is only 49.92 MB, so larger inputs are built by tiling it up to the target and trimming to whole batches

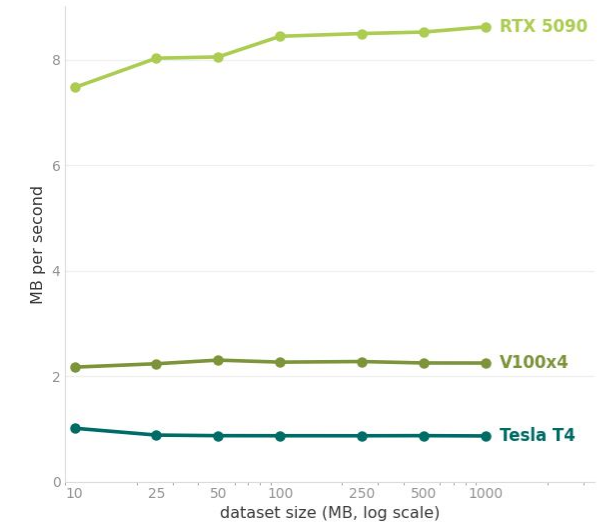
Classical baseline: the same data compressed through zstd and LZMA on a MacBook i5 CPU, into the same CSV schema.

Results

The idle floor, and how far training lifts it



Training throughput per epoch



Carbon cost scales with input size across devices

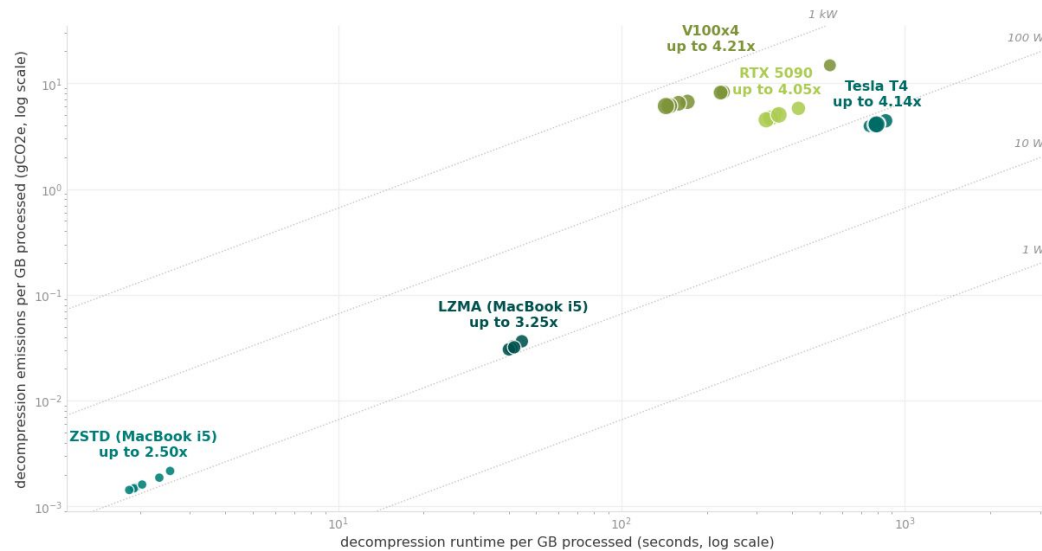


- GPUs consuming the most power
- Carbon cost linear to size of dataset
- Little effect of runtime duration
- V100 devices not parallelised hence a low throughput

Results

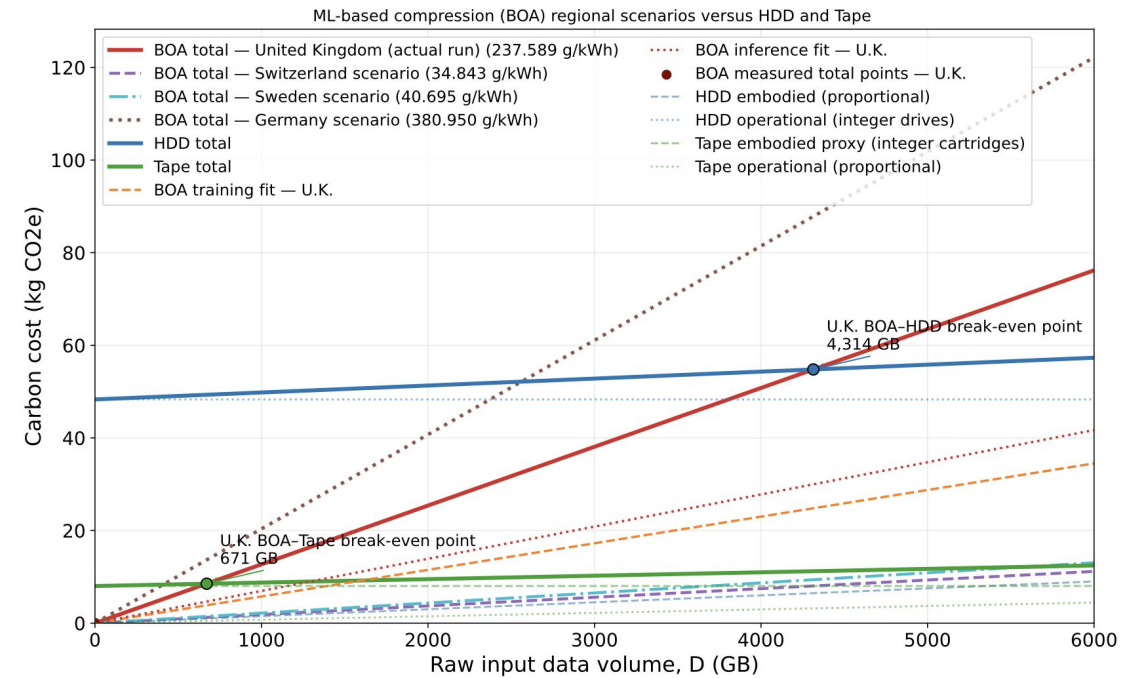
What decompression costs in runtime and carbon

One point per decompression run, normalised per GB processed. Marker size is the compression ratio achieved.



Next steps:

- The team have started mapping a pareto plot for different environments and hyperparameters
- Obtain more realistic estimates from Bede as we need to parallelise the workload better to utilise all devices
- Accelerate inference using Bit Mamba and test carbon cost



Thank you

With thanks to my supervisors **Caterina Doglioni and Akshat Gupta** and to N8CIR for hosting this internship.

References

BOA Constrictor paper

arxiv.org/abs/2511.11337

Code and learning guide

github.com/HanzilaH/N8CIR-SusTrain-data-compression-notebooks